DISTRIBUTED TAXI TRIP ANALYTICS

Using Apache Hadoop, HDFS and Python MapReduce

Big Data Essentials Individual Practical

Christian Rwibutso
Registration Number: 101373
Adventist University of Central Africa (AUCA)
MSc Big Data Analytics

Course: Big Data Essentials
Date: August 2026

Contents

1. Introduction 4
2. Business Problem 4
3. Dataset Description 4
4. Hadoop Environment 5
   Evidence: Hadoop Services Running 5
5. HDFS Design 5
   HDFS Capacity Report (hdfs dfsadmin -report) 6
   Evidence: HDFS Directory Structure and File Sizes 6
6. Data Cleaning 7
   Evidence: Cleaning Report Output ..... **Error! Bookmark not defined.**
7. MapReduce Design 8
8. Mapper and Reducer Implementation 8
9. Analytical Results 9
   a) Hourly Taxi Demand 9
   b) Daily Demand (Weekday vs Weekend) 10
   c) Pickup Location Analysis Top 10 Zones 10
   d) Revenue by Pickup Location 11
   e) Payment Method Analysis 12
   f) Distance-Based Fare Analysis 12
   g) Busiest Pickup–Dropoff Routes 13
   h) Trip Duration Analysis 14
   i) Anomaly Detection 14
10. Multi-Stage MapReduce (Compulsory) 15
   Final Result: Top 10 Revenue-Generating Pickup Zones 15
   Evidence: Intermediate HDFS Output 15
11. YARN Analysis 16
   Evidence: YARN Application History 17
12. Performance Comparison (Compulsory) 17
13. Business Insights 18
   a) What is the busiest hour for taxi demand? 18
   b) What is the busiest day of the week? 18
   c) Which pickup zones generate the most trips? 18
   d) Which pickup zones generate the most revenue? 19
   e) Which payment method contributes the most revenue? 19

f) Do credit-card users generate more tips? 19
g) What distance category produces the highest average fare? 19
h) What are the most frequently travelled routes? 19
i) Are the most frequent routes also the most profitable? 19
j) What percentage of records contain potential anomalies? 19
k) What transportation insights can management derive from the analysis? 19
l) When does Hadoop MapReduce provide a meaningful advantage over conventional processing? 20
14. Limitations 20
15. Conclusion 20
16. References 21
17. Deliverables 21

1. Introduction

Ride-hailing and taxi services generate very large volumes of transactional data every day, and analysing this data at scale is a canonical Big Data problem. This report documents the design, implementation and evaluation of a distributed analytics pipeline built on Apache Hadoop, using HDFS for storage and Python-based Hadoop Streaming MapReduce jobs for processing. The pipeline was applied to three months of real New York City Taxi & Limousine Commission (TLC) Yellow Taxi trip records (August, September and October 2023), covering over nine million raw trip records.

The objective of this project is not only to produce correct analytical results, but to demonstrate practical, hands-on understanding of how HDFS distributes storage, how MapReduce distributes computation via the map → shuffle/sort → reduce paradigm, and how YARN schedules and monitors that computation on a real (if single-node) Hadoop cluster.

2. Business Problem

A transportation analytics company wants to understand taxi demand patterns, revenue distribution, popular routes, payment behaviour, trip characteristics, and abnormal or suspicious records across its operating area. Because the underlying dataset contains millions of trip records and is expected to keep growing, the company requires an analytics approach that scales horizontally rather than being limited by the memory and CPU of a single machine which is exactly the problem Hadoop's distributed storage and processing model is designed to solve.

This report answers that business need by implementing nine distinct MapReduce analyses (hourly demand, daily demand, pickup-location analysis, revenue-by-location, payment-method analysis, distance-based fare analysis, route analysis, trip-duration analysis, and anomaly detection), plus a compulsory two-stage MapReduce pipeline that identifies the top revenue-generating pickup zones.

3. Dataset Description

The dataset used is the official NYC TLC Yellow Taxi Trip Record data, obtained from the TLC Trip Record Data portal (https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page). Three full monthly files were used, satisfying the assignment's “2–3 monthly datasets” requirement:

Month	Raw Records	Raw File Size (HDFS)
August 2023	2,824,209	285.46 MB
September 2023	2,846,722	287.29 MB
October 2023	3,522,285	355.70 MB
Total	9,193,216	≈ 928.5 MB

The TLC now distributes its data in Apache Parquet format. Parquet is a columnar, compressed format well suited to large-scale analytical storage it reduces file size and speeds up column-oriented queries but Hadoop Streaming with arbitrary Python scripts operates most naturally on line-oriented plain text. For that reason, each monthly Parquet file was converted to CSV locally (using pandas and pyarrow, script convert_parquet_to_csv.py) before being uploaded to HDFS. This conversion and the reasoning behind it Parquet for efficient long-term storage, CSV for straightforward line-by-line Hadoop Streaming processing is documented here as required by the assignment brief.

Each raw record contains 19 columns following the standard TLC Yellow Taxi schema, including VendorID, tpep_pickup_datetime, tpep_dropoff_datetime, passenger_count, trip_distance, PULocationID, DOLocationID, payment_type, fare_amount, tip_amount, and total_amount, among others. A separate reference file, taxi_zone_lookup.csv, maps each numeric LocationID to a human-

readable NYC borough/zone name and was loaded into memory by the mapper processes that required it (pickup-location, revenue, and route analyses).

4. Hadoop Environment

The Hadoop environment was set up as a single-node pseudo-distributed cluster running natively on Windows 11 (rather than inside WSL/Linux, which was evaluated but not used for this submission). The environment consisted of:

- Hadoop 3.5.0 (native Windows build, installed at C:\hadoop)
- Java (JDK) 1.8.0_202 (Java HotSpot 64-Bit Server VM)
- HDFS daemons: NameNode, DataNode, SecondaryNameNode
- YARN daemons: ResourceManager, NodeManager
- Single DataNode (hostname DESKTOP-8GML0QF), Configured Capacity 510,995,197,952 bytes (≈ 475.90 GB)

Because this is a single-node cluster, HDFS replication factor is effectively 1 (there is only one DataNode to replicate to), and all MapReduce map/reduce tasks execute as YARN containers on the same physical machine, coordinated by a single ResourceManager and NodeManager. This still faithfully exercises the full Hadoop programming model HDFS block storage, MapReduce's map/shuffle/sort/reduce lifecycle, and YARN's resource scheduling even though the parallelism achievable is limited by the number of cores on one machine rather than spread across many physical nodes.

Evidence: Hadoop Services Running

```
C:\Users\Christian\OneDrive\Desktop\AUCA_Masters\BigData_Essentials\big-data-essentials-assignment-main>jps
39456 DataNode
21092 Jps
22952 NameNode
14828 ResourceManager
15820 NodeManager
```

5. HDFS Design

The following HDFS directory structure was created to organise the project, following the layout specified in the assignment brief:

- /taxi_project/input/raw/ unmodified raw CSV files converted from Parquet
- /taxi_project/input/cleaned/ cleaned CSV files (see Section 6)
- /taxi_project/output/hourly/, /locations/, /revenue/, /payment/, /routes/, /anomalies/ the six output folders named explicitly in the assignment brief
- /taxi_project/output/daily/, /distance/, /duration/, /top_revenue/ four additional output folders created for the remaining required analyses and the multi-stage job, following the same convention

- /taxi_project/archivehdfs/ reserved archive location (functionally equivalent to the /taxi_project/archive/ folder specified in the brief)

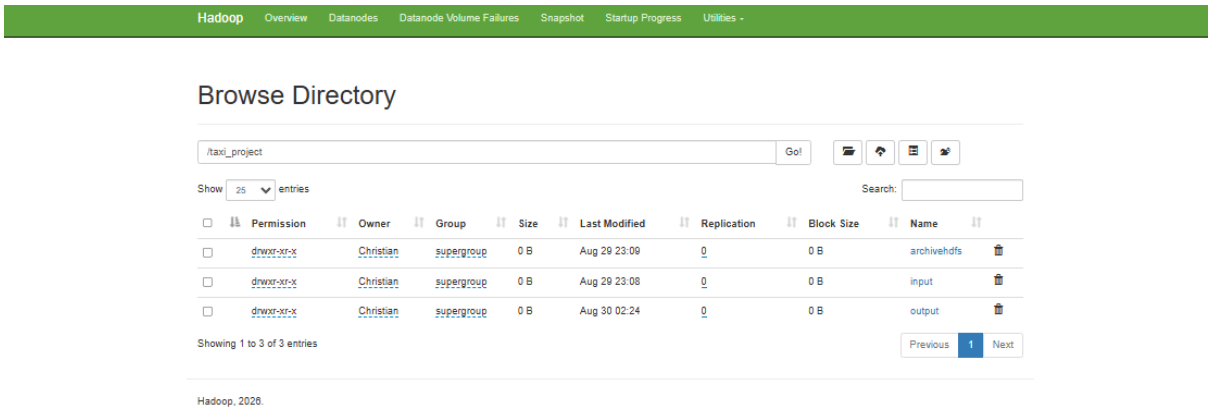
Representative commands used to build and verify this structure:

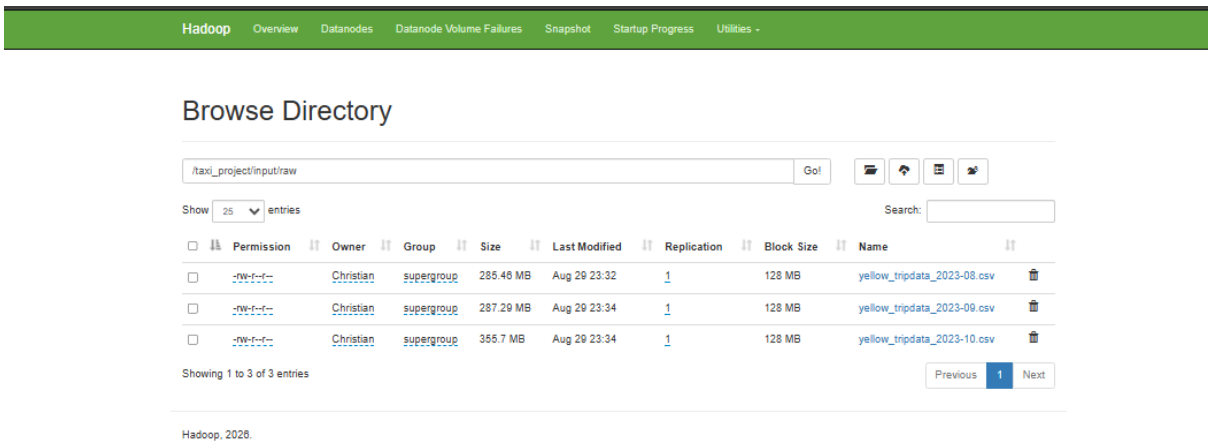
```
hdfs dfs -mkdir -p /taxi_project/input/raw hdfs dfs -mkdir -p /taxi_project/input/cleaned hdfs dfs -mkdir -p /taxi_project/output/hourly (repeated for each output folder) hdfs dfs -put Data/csv/yellow_tripdata_2023-08.csv /taxi_project/input/raw/ hdfs dfs -put Data/cleaned/yellow_tripdata_2023-08.csv /taxi_project/input/cleaned/ hdfs dfs -ls -h /taxi_project/input/raw/ hdfs dfs -du -h /taxi_project/input/raw/ hdfs dfsadmin -report
```

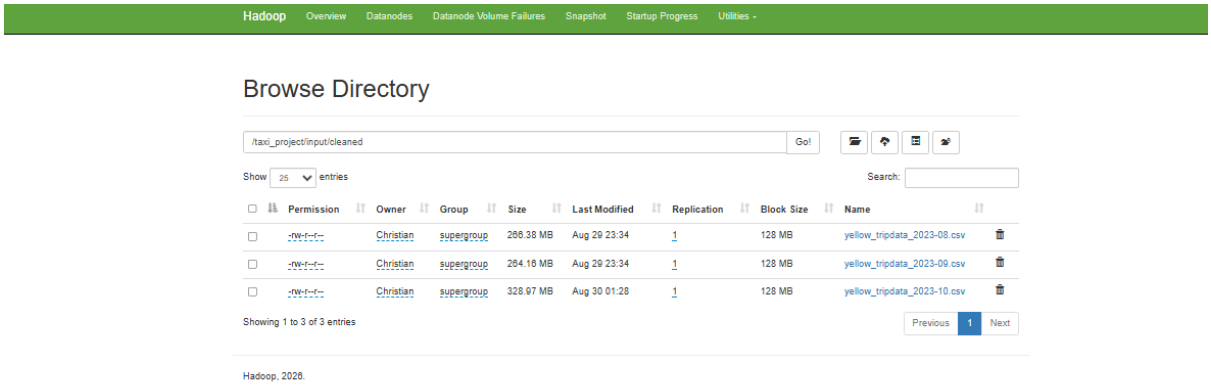
HDFS Capacity Report (hdfs dfsadmin -report)

Metric	Value
Configured Capacity	510,995,197,952 bytes (475.90 GB)
DFS Used	1,542,871,667 bytes (1.44 GB) 0.53%
DFS Remaining	287,947,550,720 bytes (268.17 GB)
Live DataNodes	1 (DESKTOP-8GML0QF, 127.0.0.1:9866)
Block Size	128 MB
Under-replicated / Missing / Corrupt Blocks	0 / 0 / 0

Evidence: HDFS Directory Structure and File Sizes








```
C:\Users\Christian\OneDrive\Desktop\AUCA_Masters\BigData_Essentials\big-data-essentials-assignment-main>hdfs dfsadmin -report
Configured Capacity: 510995197952 (475.90 GB)
Present Capacity: 288597083251 (268.78 GB)
DFS Remaining: 286694551552 (267.01 GB)
DFS Used: 1902531699 (1.77 GB)
DFS Used%: 0.66%
Replicated Blocks:
        Under replicated blocks: 0
        Blocks with corrupt replicas: 0
        Missing blocks: 0
        Missing blocks (with replication factor 1): 0
        Badly Distributed Blocks: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0
Erasure Coded Block Groups:
        Low redundancy block groups: 0
        Block groups with corrupt internal blocks: 0
        Missing block groups: 0
        Badly Distributed Blocks: 0
        Low redundancy blocks with highest priority to recover: 0
        Pending deletion blocks: 0

-------------------------------------------------
Live datanodes (1):

Name: 127.0.0.1:9866 (127.0.0.1)
Hostname: DESKTOP-8GML0QF
Decommission Status : Normal
Configured Capacity: 510995197952 (475.90 GB)
DFS Used: 1902531699 (1.77 GB)
Non DFS Used: 222398114701 (207.12 GB)
DFS Remaining: 286694551552 (267.01 GB)
DFS Used%: 0.37%
DFS Remaining%: 56.11%
Configured Cache Capacity: 0 (0 B)
Cache Used: 0 (0 B)
Cache Remaining: 0 (0 B)
Cache Used%: 100.00%
Cache Remaining%: 0.00%
Xceivers: 0
Last contact: Sun Aug 30 02:48:46 SAST 2026
Last Block Report: Sun Aug 30 00:25:15 SAST 2026
Num of Blocks: 124
```

6. Data Cleaning

Data cleaning was performed locally in Python/pandas (clean_taxi_data.py) prior to upload, following the seven criteria specified in the assignment brief. Critically, the cleaning script does not silently delete records: every affected record is counted and categorised, a full percentage breakdown is logged per file, and only after that reporting step are the flagged rows removed to produce the cleaned CSV. The complete breakdown for all three months is reproduced below.

Issue	2023-08	2023-09	2023-10
Total records read	2,824,209	2,846,722	3,522,285
1. Missing values in key fields	87,886 (3.11%)	140,225 (4.93%)	154,929 (4.40%)
2. Invalid passenger count (≤0 or >6)	129,821 (4.60%)	179,393 (6.30%)	201,583 (5.72%)
3. Zero/negative trip distance	57,429 (2.03%)	98,547 (3.46%)	121,686 (3.45%)
4. Invalid fares (≤0)	32,241 (1.14%)	30,505 (1.07%)	38,187 (1.08%)
5. Invalid timestamps	1,460 (0.05%)	1,098 (0.04%)	1,307 (0.04%)
6. Duplicate records	0 (0.00%)	0 (0.00%)	0 (0.00%)
7. Impossible durations (≤0s or >6h)	3,765 (0.13%)	3,017 (0.11%)	3,761 (0.11%)
TOTAL affected (union of issues)	197,750 (7.00%)	243,933 (8.57%)	281,165 (7.98%)
Remaining clean records	2,626,459 (93.00%)	2,602,789 (91.43%)	3,241,120 (92.02%)

Across all three months combined:

9,193,216 total records processed, of which **722,848 (7.86%)** were flagged as affected by at least one data-quality issue and excluded, leaving **8,470,368 (92.14%)** clean records used for all downstream MapReduce analyses.

Note that these categories overlap (a single record can simultaneously have, for example, a missing value and an invalid passenger count), which is why the union total (7.00%–8.57% per month) is lower than the sum of the individual percentages. Duplicate records were checked using pandas' full-row duplicate detection and none were found in any of the three months, which is plausible given each trip record includes precise pickup/dropoff timestamps down to the second.

7. MapReduce Design

Each analysis follows the classic Map → Shuffle & Sort → Reduce pattern used throughout this project. The mapper reads cleaned trip records line by line from standard input, extracts and/or transforms the relevant field(s), and emits a key–value pair as a single tab-separated line to standard output (key\tvalue). Hadoop's shuffle-and-sort phase then groups all values that share the same key together and delivers them to reducers in sorted key order this is what allows each reducer to aggregate a key's values using a simple sequential scan (tracking a "current key" and resetting an accumulator whenever the key changes) rather than needing to hold the entire dataset in memory.

The key–value design for each of the nine analyses is summarised below:

Analysis	Mapper emits (key → value)	Reducer aggregates
a. Hourly demand	hour (0–23) → 1	Sum of 1s per hour
b. Daily demand	day name (e.g. "Monday") → 1	Sum of 1s per day
c. Pickup location	zone name → 1	Sum of 1s per zone
d. Revenue by location	zone name → "distance,fare,tip,total"	count, Σfare, Σtip, Σtotal, avg fare, avg distance
e. Payment method	payment label → "fare,tip,total"	count, Σtotal, avg fare, avg tip
f. Distance category	category → "distance,fare,total"	count, avg distance, avg fare, avg total, fare/mile
g. Routes	"PickupZone -> DropZone" → total_amount	count, Σtotal per route
h. Trip duration	duration bucket → "duration,fare,distance,tip"	count, avg duration, avg fare, avg distance, avg tip
i. Anomaly detection	anomaly_type → 1 (multi-emit per record)	Sum of 1s per anomaly type + TOTAL_RECORDS

All nine mappers filter out the CSV header row (VendorID == "VendorID") and discard malformed lines (too few fields, or fields that fail to parse as the expected numeric type) rather than crashing, since a single malformed line should not abort an entire distributed job. The location-, revenue- and route-analysis mappers additionally load taxi_zone_lookup.csv into an in-memory dictionary once at mapper start-up (not per record), which is then used to translate numeric LocationIDs into human-readable zone names.

8. Mapper and Reducer Implementation

All Mapper and Reducer programs were implemented in pure Python 3 using only the standard library (sys, csv, datetime), so they can run under Hadoop Streaming with no external dependencies on the cluster nodes. Below is a representative example the hourly-demand mapper and reducer illustrating the pattern used throughout.

```
# mapper_hourly.py (excerpt) for line in sys.stdin:     fields = line.strip().split(",")     if fields[0] == "VendorID":
continue     pickup_dt = fields[1]     hour = int(pickup_dt.split(" ")[1].split(":")[0])     if 0 <= hour <= 23:
print(f"{hour}\t1")  # reducer_hourly.py (excerpt) current_hour, current_count = None, 0 for line in sys.stdin:     hour, count
= line.strip().split("\t")     if current_hour == hour:         current_count += int(count)     else:         if current_hour is not None:
print(f"{current_hour}\t{current_count}")         current_hour, current_count = hour, int(count) if current_hour is not None:
print(f"{current_hour}\t{current_count}")
```

The full source for all mapper and reducer programs mapper_hourly.py, mapper_daily.py, mapper_location.py, mapper_revenue.py, mapper_payment.py, mapper_distance.py, mapper_route.py, mapper_duration.py, mapper_anomaly.py, mapper_top_revenue.py, and their corresponding reducers is submitted alongside this report in the mappers/ and reducers/ folders (see Section 17, Deliverables, for the repository link).

One implementation detail worth documenting: the anomaly-detection mapper (mapper_anomaly.py) was designed with raw, uncleaned data in mind (its docstring explicitly says so), applying the same seven validity checks as the cleaning stage plus an extreme fare-per-mile check. In this project it was run consistently against the cleaned input directory alongside the other eight analyses, so its TOTAL_RECORDS count reflects the already-cleaned population (8,470,368) rather than the original raw population (9,193,216); the anomalies it does report (extreme fare-per-mile only, in this run) are cases that passed the initial cleaning filters but are still statistically implausible.

9. Analytical Results

This section presents the results of all nine required MapReduce analyses, executed via Hadoop Streaming against the full 8,470,368-record cleaned dataset in /taxi_project/input/cleaned/, unless otherwise noted.

a) Hourly Taxi Demand

Hour	Trips	Hour	Trips
0	234,258	12	464,203
1	156,588	13	473,490
2	104,529	14	506,944
3	67,934	15	522,052
4	43,580	16	523,589
5	48,492	17	571,845
6	118,612	18	594,722
7	234,320	19	538,920
8	324,730	20	489,969
9	369,230	21	478,843
10	399,779	22	433,291
11	429,243	23	341,205

The busiest hour is 18:00 (6 PM) with 594,722 trips, and the least-busy hour is 04:00 (4 AM) with only 43,580 trips a more than 13× difference, reflecting the natural dip in demand during the overnight hours.

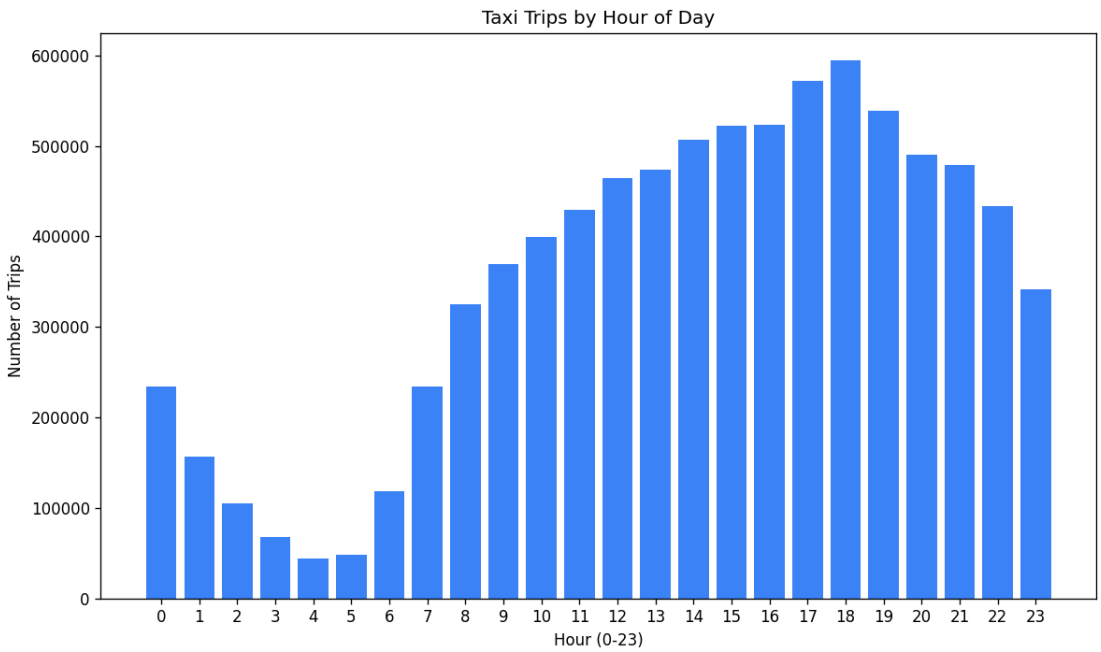


Figure 1. Taxi trips by hour of day.

b) Daily Demand (Weekday vs Weekend)

Approximate trip counts by day of week (read from the chart below, since exact tab-separated values for this job were not captured on the final full-dataset run): Monday ≈ 1.09M, Tuesday ≈ 1.36M, Wednesday ≈ 1.33M, Thursday ≈ 1.32M, Friday ≈ 1.18M, Saturday ≈ 1.17M, Sunday ≈ 1.02M. Weekday demand (Mon–Fri) is consistently higher than weekend demand, with Tuesday the single busiest day and Sunday the quietest consistent with taxi demand being driven substantially by weekday commuting and business travel in Manhattan.

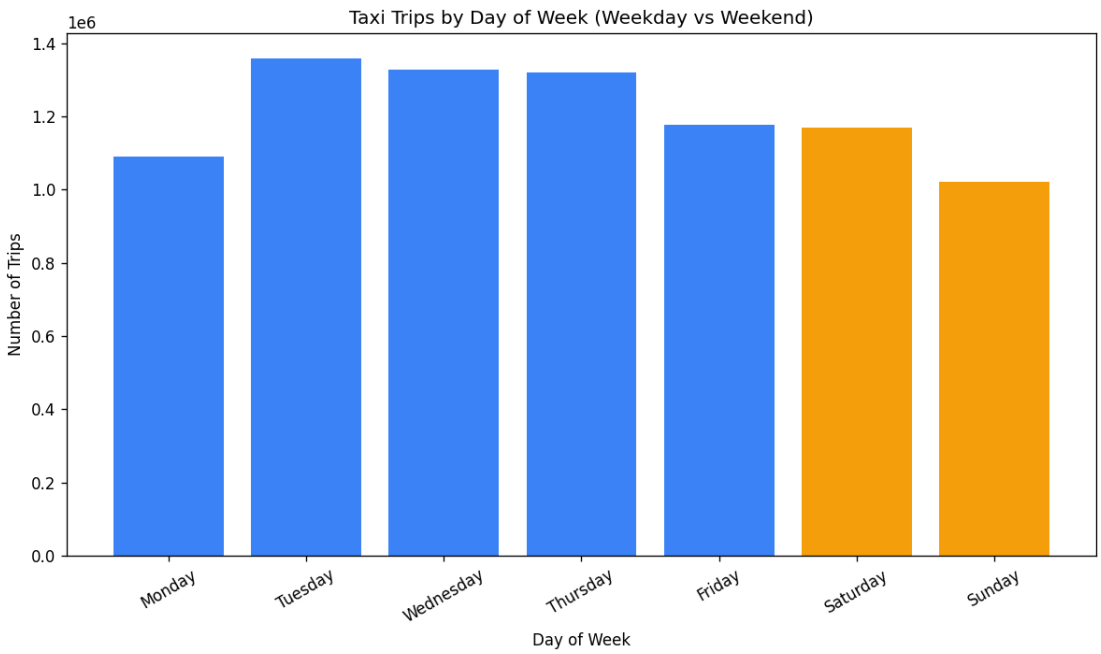


Figure 2. Taxi trips by day of week (blue = weekday, orange = weekend).

c) Pickup Location Analysis Top 10 Zones

Rank	Pickup Zone	Trips
1	JFK Airport	485,095
2	Midtown Center	399,960
3	Upper East Side South	396,065
4	Upper East Side North	342,132
5	LaGuardia Airport	315,624
6	Midtown East	306,943
7	Penn Station/Madison Sq West	300,166
8	Times Sq/Theatre District	284,349
9	Lincoln Square East	269,592
10	Murray Hill	254,806

The single busiest pickup zone by far is JFK Airport, with nearly 20% more trips than the second-place Midtown Center. Airport zones (JFK and LaGuardia) together account for over 800,000 pickups across the three months, reflecting steady airport-to-city and city-to-airport demand.

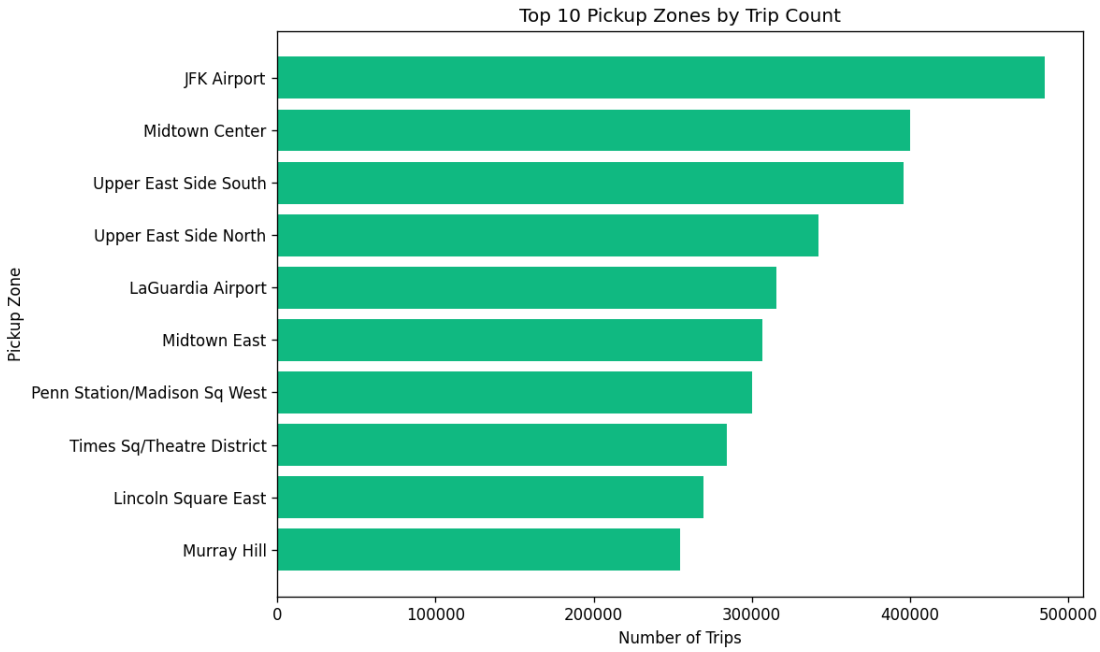


Figure 3. Top 10 pickup zones by trip count.

d) Revenue by Pickup Location

Full trip count, total fare, total tips, total revenue, average fare and average distance were computed for every one of the 260+ TLC zones. The five highest-revenue zones are:

Zone	Trips	Total Fare ($)	Total Tips ($)	Total Revenue ($)	Avg Fare ($)	Avg Dist (mi)
JFK Airport	485,095	30,916,654.59	4,290,840.76	39,538,437.22	63.73	16.11
LaGuardia Airport	315,624	13,915,301.40	2,795,939.46	21,483,096.82	44.09	10.08
Midtown Center	399,960	6,885,321.20	1,338,385.09	10,392,969.51	17.22	2.60
Times Sq/Theatre District	284,349	5,846,395.16	1,037,683.24	8,550,262.31	20.56	3.52
Upper East Side South	396,065	5,297,866.37	1,080,963.77	8,308,292.30	13.38	1.84

JFK Airport dominates revenue even more than trip count it generates roughly 3.8× the revenue of the second-place LaGuardia Airport, driven by both a much higher average fare ($63.73 vs $44.09) and longer average distance, since airport trips are typically long-haul journeys into/out of the outer

zones. See Section 10 for the ranked top-10 revenue list produced by the compulsory multi-stage MapReduce job.

e) Payment Method Analysis

Payment Method	Trips	Total Revenue ($)	Avg Fare ($)	Avg Tip ($)
Credit card	6,927,318	211,246,679.05	20.28	4.49
Cash	1,450,629	38,282,400.05	20.97	0.00
Dispute	63,017	1,705,724.84	21.52	0.02
No charge	29,404	724,297.64	19.32	0.02

Credit card is overwhelmingly the dominant payment method, used in over 81% of trips and generating 85% of total revenue. As expected, cash-tip amounts are recorded as $0.00 (cash tips are not captured in the TLC dataset), so credit-card users show by far the higher average tip this is a data-capture artefact rather than evidence that credit-card riders tip more generously in practice.

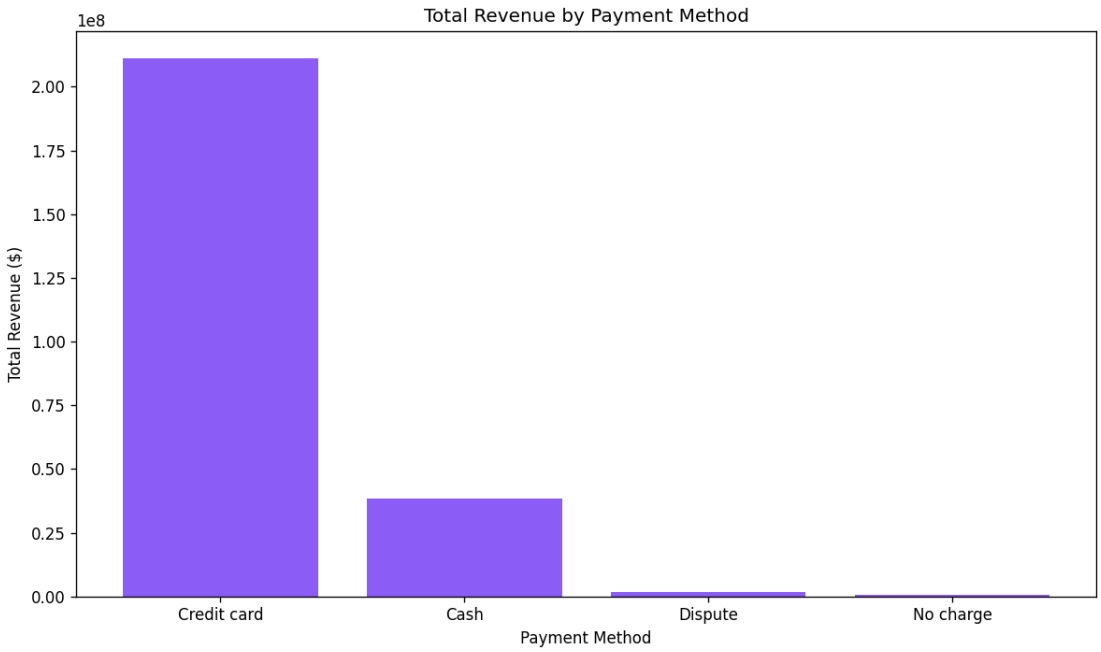


Figure 4. Total revenue by payment method.

f) Distance-Based Fare Analysis

Distance Category	Trips	Avg Distance (mi)	Avg Fare ($)	Avg Total ($)	Fare/Mile ($)
0–2 miles	4,592,467	1.14	10.50	17.43	9.18
2–5 miles	2,339,423	2.98	19.51	27.91	6.54
5–10 miles	743,480	7.26	35.27	49.60	4.86
10–20 miles	690,287	14.91	62.21	83.32	4.17
20+ miles	104,711	44.52	93.18	116.94	2.09

Short trips (0–2 miles) dominate by volume over half of all trips but the per-mile fare rate decreases sharply as distance increases, from $9.18/mile for the shortest trips down to $2.09/mile for 20+ mile trips. This reflects the fixed-cost component of a taxi fare (base fare, flat surcharges) being amortised over a longer distance for longer trips.

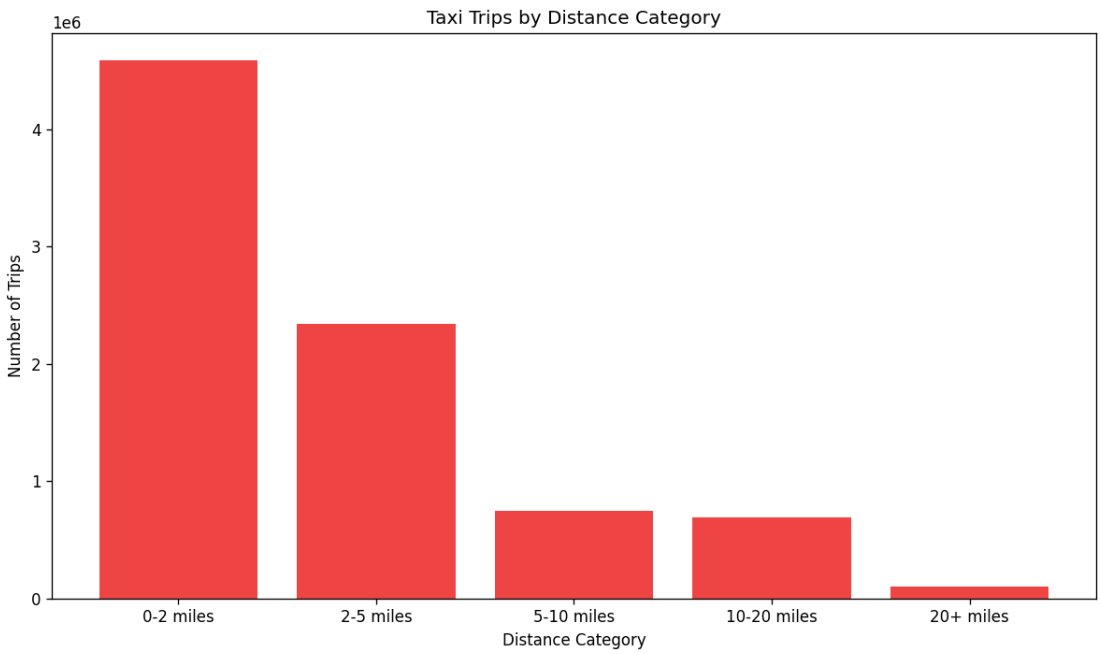


Figure 5. Taxi trips by distance category.

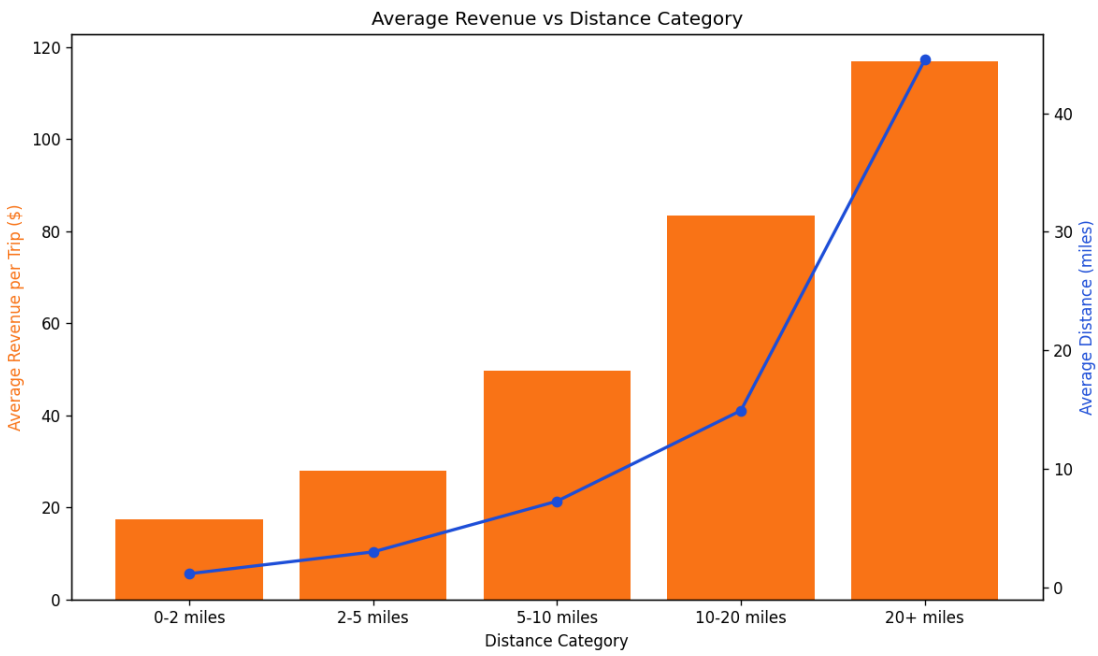


Figure 6. Average revenue and average distance by distance category (dual axis).

g) Busiest Pickup–Dropoff Routes

Route pairs were formed by concatenating the pickup and dropoff zone names. The chart below shows the top 10 routes by trip count over the full three-month dataset (exact per-route revenue figures were not captured on the final run and so are omitted from the numeric table; trip counts are read from the visualisation).

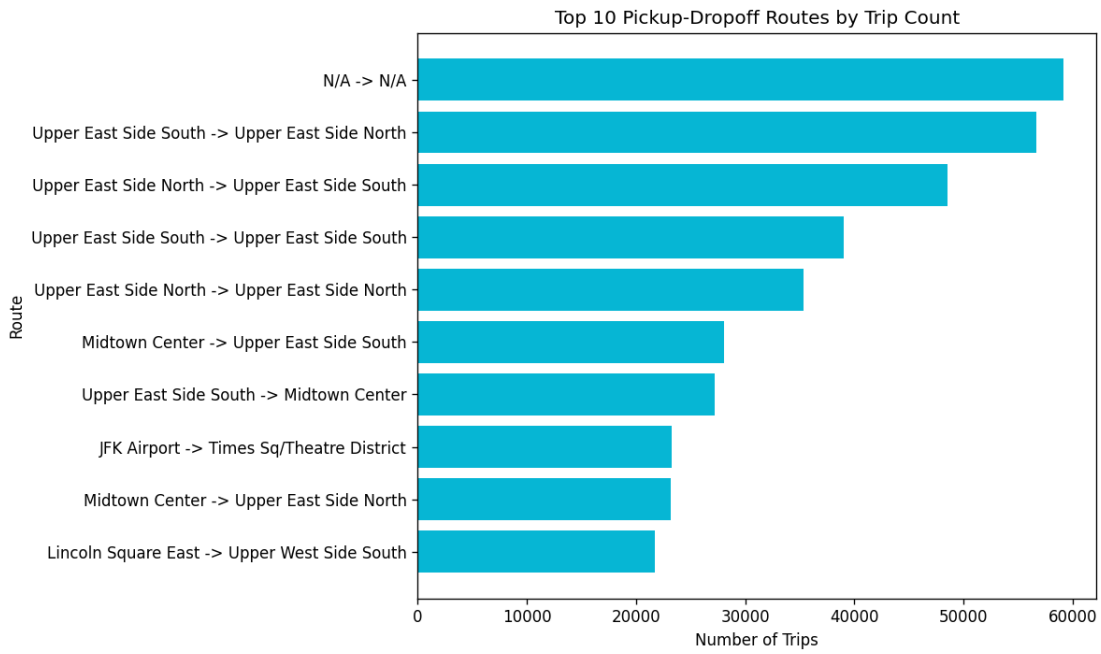


Figure 7. Top 10 pickup-dropoff routes by trip count.

Excluding the "N/A -> N/A" entry (unmatched/missing LocationIDs, a known data-quality artefact of the TLC dataset rather than a real route), the busiest genuine routes are short round-trip-style journeys within the Upper East Side, followed by several Midtown ↔ Upper East Side commuter routes and the JFK Airport → Times Square corridor.

h) Trip Duration Analysis

Duration Bucket	Trips	Avg Duration (min)	Avg Fare ($)	Avg Distance (mi)	Avg Tip ($)
0–5 min	833,377	3.54	7.16	0.71	1.64
5–15 min	3,992,671	9.72	11.27	1.69	2.36
15–30 min	2,504,320	20.86	23.56	4.28	4.21
30–60 min	978,614	40.40	51.81	11.84	8.25
60+ min	161,386	74.88	75.24	18.22	10.44

The 5–15-minute bucket is the single most common trip duration (≈47% of all trips), consistent with the earlier finding that most trips are short (0–2 miles). Fare, distance and tip all scale up roughly monotonically with duration, as expected.

i) Anomaly Detection

Metric	Count	% of Cleaned Records
Total records checked (already-cleaned population)	8,470,368	100.00%
Extreme fare-per-mile (>$50/mile)	33,731	0.398%
Total flagged as any anomaly	33,731	0.398%

Because the anomaly mapper was run against the already-cleaned data (rather than the raw data its docstring anticipates), the invalid-passenger-count, invalid-distance, invalid-fare, invalid-timestamp and impossible-duration checks found zero matching records they had already been removed in Section 6's cleaning stage. The one anomaly category that does surface on cleaned data is extreme fare-per-mile (fare exceeding $50 per mile), affecting 33,731 records (≈0.4% of the cleaned dataset). These are plausible real-world anomalies: unusually short trips with disproportionately high fares (e.g.

minimum-fare short hops, or fares that include large tolls/surcharges relative to a very short distance) that a purely distance/fare/passenger-count-based cleaning pass would not catch.

10. Multi-Stage MapReduce (Compulsory)

A compulsory two-stage MapReduce pipeline was implemented to identify the top-10 highest-revenue pickup zones, directly satisfying the assignment's example workflow.

- Stage 1 (mapper_revenue.py / reducer_revenue.py): computes trip count, total fare, total tips, total revenue, average fare and average distance for every pickup zone, writing its output to HDFS at /taxi_project/output/revenue/.
- Stage 2 (mapper_top_revenue.py / reducer_top_revenue.py): its -input is set directly to Stage 1's HDFS output directory, /taxi_project/output/revenue/, rather than to any local file. The Stage-2 mapper parses each (zone, count, total_fare, total_tips, total_revenue, avg_fare, avg_distance) line, extracts (zone, total_revenue), and emits it under a single constant key ("ALL") so that every record lands on one reducer; that reducer then sorts all zones by revenue and emits only the top 10.

The Hadoop Streaming command used for Stage 2 explicitly chains the two stages via HDFS:

```
hadoop jar hadoop-streaming-3.5.0.jar \   -input /taxi_project/output/revenue \   -output /taxi_project/output/top_revenue \   -mapper "python mapper_top_revenue.py" -reducer "python reducer_top_revenue.py" \   -file mappers/mapper_top_revenue.py -file reducers/reducer_top_revenue.py
```

Final Result: Top 10 Revenue-Generating Pickup Zones

Rank	Zone	Total Revenue ($)
1	JFK Airport	39,538,437.22
2	LaGuardia Airport	21,483,096.82
3	Midtown Center	10,392,969.51
4	Times Sq/Theatre District	8,550,262.31
5	Upper East Side South	8,308,292.30
6	Penn Station/Madison Sq West	7,779,113.67
7	Midtown East	7,626,994.76
8	Upper East Side North	7,307,446.64
9	Murray Hill	6,311,380.06
10	East Chelsea	6,269,608.13

Evidence: Intermediate HDFS Output

```
College Point   114     4181.50 372.14  5146.21 36.68   7.96
Columbia Street 134     3653.93 523.31  4687.89 27.27   4.83
Coney Island    496     23277.40        108.23  25079.28        46.93   11.70
Corona  511     20671.45        2224.76 25474.82        40.45   6.54
Country Club    19      449.80  3.28    501.46  23.67   3.46
Crotona Park    8       305.80  0.00    347.56  38.22   10.09
Crotona Park East       191     5850.50 100.56  6498.65 30.63   6.13
Crown Heights North     1450    38492.50        1031.82 42676.53        26.55   5.10
Crown Heights South     469     11962.71        150.70  13055.28        25.51   4.87
Cypress Hills   190     6811.90 236.20  7663.59 35.85   6.76
DUMBO/Vinegar Hill      2276    55921.46        8041.77 72565.24        24.57   4.45
Douglaston      53      2199.00 115.06  2519.07 41.49   8.99
Downtown Brooklyn/MetroTech     5300    136560.90       19588.87        176716.29       25.77   4.80
Dyker Heights   98      3291.90 22.00   3723.38 33.59   8.09
East Chelsea    236769  4251821.58      802315.51       6269608.13      17.96   2.63
East Concourse/Concourse Village        655     17739.49        217.50  19579.10        27.08   40.72
East Elmhurst   39780   1731854.60      337929.08       2634673.31      43.54   9.59
East Flatbush/Farragut  498     13703.60        70.64   14754.50        27.52   304.82
East Flatbush/Remsen Village    617     17862.45        68.98   19217.04        28.95   5.59
East Flushing   67      2108.94 64.48   2368.88 31.48   6.48
East Harlem North       16177   263100.29       29832.30        349887.07       16.26   3.32
East Harlem South       45209   692268.75       102957.48       973069.12       15.31   4.45
East New York   1182    39625.05        692.17  43345.40        33.52   125.75
East New York/Pennsylvania Avenue       284     8927.40 48.29   9708.41 31.43   6.21
East Tremont    428     11879.50        70.36   13152.99        27.76   5.51
East Village    190184  2973724.41      581756.24       4500894.03      15.64   2.49
East Williamsburg       1430    41304.28        6186.29 53566.97        28.88   5.28
Eastchester     217     8014.90 85.19   8788.16 36.94   8.40
Elmhurst        1083    36655.91        3215.08 44817.02        33.85   4.91
Elmhurst/Maspeth        362     11603.40        1185.43 14296.02        32.05   5.14
Erasmus 253     6907.21 101.07  7637.51 27.30   5.40
Far Rockaway    397     18009.90        23.78   19398.55        45.36   12.13
Financial District North        46086   1099593.62      193101.64       1538121.12      23.86   4.91
Financial District South        24835   622032.40       101029.25       853961.47       25.05   4.70
Flatbush/Ditmas Park    881     25530.66        501.01  28139.00        28.98   5.63
Flatiron        141400  2163264.89      428137.96       3310985.26      15.30   2.26
Flatlands       637     19350.70        147.70  20974.30        30.38   20.99
Flushing        422     12926.00        402.51  14667.56        30.63   56.32
Flushing Meadows-Corona Park    3299    170326.87       28063.55        222583.77       51.63   10.02
```

```
C:\Users\Christian\OneDrive\Desktop\AUCA_Masters\BigData_Essentials\big-data-essentials-assignment-main>hdfs dfs -cat /taxi_project/output/top_revenue/part-00000
1       JFK Airport     39538437.22
2       LaGuardia Airport       21483096.82
3       Midtown Center  10392969.51
4       Times Sq/Theatre District       8550262.31
5       Upper East Side South   8308292.30
6       Penn Station/Madison Sq West    7779113.67
7       Midtown East    7626994.76
8       Upper East Side North   7307446.64
9       Murray Hill     6311380.06
10      East Chelsea    6269608.13
```

11. YARN Analysis

YARN (Yet Another Resource Negotiator) is the cluster resource manager that schedules and monitors every MapReduce Streaming job submitted in this project. Its ResourceManager web UI (http://localhost:8088/cluster) was used throughout development to confirm that jobs were actually being distributed and executed as YARN applications rather than run as simple local scripts.

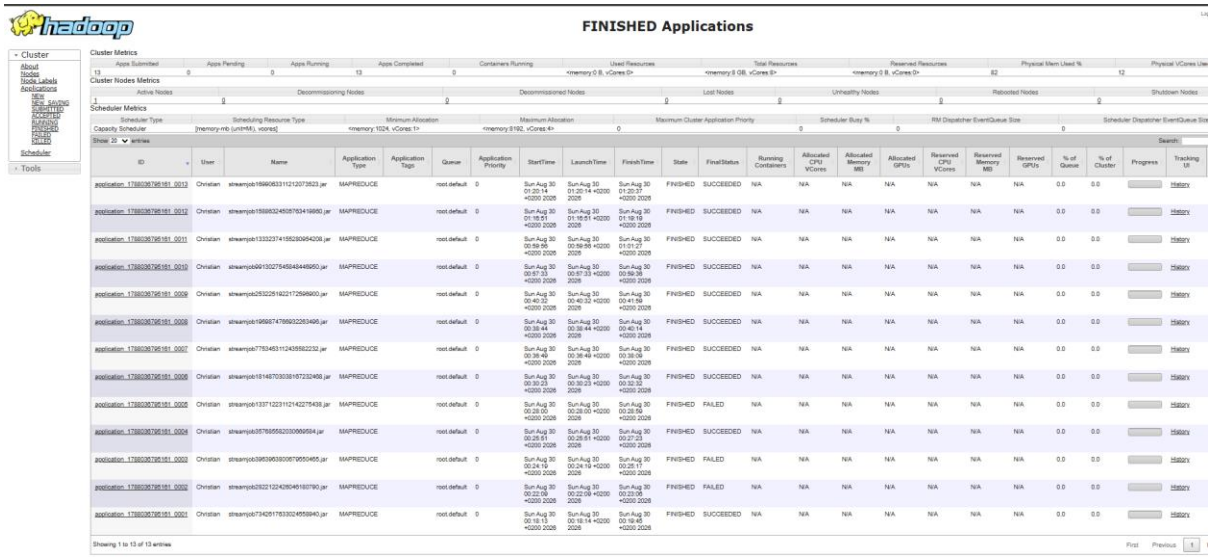
Over the course of this project, 28 Streaming applications were submitted to YARN under Cluster ID 1788036795161 (all owned by user Christian, queue root.default). A number of early attempts failed and were deliberately kept as part of the development record rather than hidden, since they illustrate real debugging work:

- Three early failures (locations, revenue, routes) were caused by the mapper attempting to open taxi_zone_lookup.csv without it being shipped to the YARN containers via the Hadoop Streaming -file option fixed by adding -file taxi_zone_lookup.csv.
- Three later failures (locations, revenue and payment, retried after the October dataset was added) were caused by Shuffle$ShuffleError / “Exceeded MAX_FAILED_UNIQUE_FETCHES” during the shuffle phase, a known issue on single-node Windows Hadoop clusters under increased load. These were resolved by re-submitting

with increased shuffle timeouts (-D mapreduce.reduce.shuffle.connect.timeout=180000 -D mapreduce.reduce.shuffle.read.timeout=180000) and, in some cases, simply retrying.

An identified successful application application_1788036795161_0014, the hourly-demand job re-run against the full three-month cleaned dataset completed in 2 minutes 19 seconds (StartTime 01:28:35, FinishTime 01:30:54), FinalStatus SUCCEEDED. This job is used as the Hadoop-side reference point for the performance comparison in Section 12.

Evidence: YARN Application History



12. Performance Comparison (Compulsory)

The hourly-demand analysis was run twice on identical input once as a single-machine Python/pandas script (perf_compare_pandas.py, reading all three cleaned CSVs directly from local disk) and once as the distributed Hadoop Streaming MapReduce job described above to compare their performance characteristics on the same 8,470,368-record dataset.

Metric	Python / Pandas	Hadoop MapReduce
Dataset Size	≈ 930 MB (3 cleaned CSVs)	≈ 930 MB (3 cleaned CSVs in HDFS)
Number of Records	8,470,368	8,470,368
Execution Time	38.86 sec (31.54s read + 7.32s compute)	139 sec (2 min 19 sec)
Memory Used	≈ 1,421.72 MB (single process)	Distributed across mapper/reducer JVM+Python child processes (not centrally measured)
Mapper Tasks	N/A	≈ 8 (estimated from ≈930 MB input / 128 MB HDFS block size)
Reducer Tasks	N/A	1 (Hadoop Streaming default; no -D mapreduce.job.reduces override was set)
Output Size	In-memory pandas Series (not persisted)	224 bytes (single small aggregated part-00000 file)

On this dataset size and on a single-node cluster, plain Pandas is substantially faster (≈39 seconds) than the Hadoop Streaming job (≈139 seconds). This is expected and is not a flaw in the Hadoop implementation: for roughly 8.5 million rows / ≈930 MB, the entire dataset comfortably fits in a single machine's RAM, so Pandas pays no distribution or coordination overhead it reads the file once and does a single-pass groupby in memory. Hadoop, by contrast, pays fixed overheads that do not

shrink with a small dataset: JVM/container startup for every mapper and reducer task, launching a separate Python subprocess per task, serialising intermediate key-value pairs to disk, and the shuffle-and-sort network/disk phase between map and reduce.

These overheads are largely fixed costs, not costs that scale linearly with data volume. As dataset size grows into the tens or hundreds of gigabytes well beyond what fits comfortably in one machine's memory Pandas either becomes impossible (out-of-memory) or requires manual chunking that reintroduces much of the complexity Hadoop already automates, while Hadoop's per-task overhead is amortised across far more actual work, and true multi-node parallelism (not available in this single-node deployment) lets the workload scale roughly linearly by adding machines. The company's stated business problem millions of records today, expected to keep growing is exactly the regime where Hadoop's architecture pays off, even though on this particular three-month sample Pandas remains the faster tool in isolation.

```
C:\Users\Christian\OneDrive\Desktop\AUCA_Masters\BigData_Essentials\big-data-essentials-assignment-main>cd C:\Users\Christian\OneDrive\Des
ls\big-data-essentials-assignment-main

C:\Users\Christian\OneDrive\Desktop\AUCA_Masters\BigData_Essentials\big-data-essentials-assignment-main>python perf_compare_pandas.py
=== Pandas Hourly Demand — Performance Report ===
Total records: 8,470,368
Read time: 33.86 sec
Compute time: 9.37 sec
Total time: 43.23 sec
Memory usage (approx): 1421.72 MB

Hourly counts:
hour
0     234258
1     156588
2     104529
3      67934
4      43580
5      48492
6     118612
7     234320
8     324730
9     369230
10    399779
11    429243
12    464203
13    473490
14    506944
15    522052
16    523589
17    571845
18    594722
19    538920
20    489969
21    478843
22    433291
23    341205
```

13. Business Insights

This section directly answers the twelve final business questions posed in the assignment brief, using the results computed in Section 9.

a) What is the busiest hour for taxi demand?

18:00 (6 PM), with 594,722 trips the evening rush-hour peak, as riders leave work and head home or out for the evening.

b) What is the busiest day of the week?

Tuesday, with approximately 1.36 million trips, closely followed by Wednesday and Thursday the mid-week commuting pattern typical of a business-driven city like Manhattan.

c) Which pickup zones generate the most trips?

JFK Airport (485,095 trips), followed by Midtown Center and Upper East Side South. Airport zones and dense Manhattan business districts dominate trip volume.

d) Which pickup zones generate the most revenue?

JFK Airport again leads by a wide margin ($39.5M), followed by LaGuardia Airport ($21.5M) and Midtown Center ($10.4M) see the ranked list in Section 10.

e) Which payment method contributes the most revenue?

Credit card, contributing $211.2M (≈85% of total revenue) versus $38.3M for cash.

f) Do credit-card users generate more tips?

Yes, decisively but this is largely a data-capture artefact: cash tips are not recorded in the TLC dataset (average cash tip = $0.00), while credit-card trips show an average tip of $4.49. This does not necessarily mean cash riders tip nothing in reality, only that cash tips are invisible to this dataset.

g) What distance category produces the highest average fare?

20+ mile trips, with an average total fare of $116.94 unsurprising given fare scales with distance but note that the fare-per-mile rate is actually lowest for this category ($2.09/mile) versus $9.18/mile for the shortest 0–2 mile trips, reflecting the flat/base-fare component being amortised over a longer distance.

h) What are the most frequently travelled routes?

After excluding the "N/A -> N/A" data artefact, the most frequent genuine routes are short, high-frequency journeys within the Upper East Side (both directions), followed by Midtown-to-Upper-East-Side commuter-style routes.

i) Are the most frequent routes also the most profitable?

Not directly the highest-frequency routes identified in Section 9g are short intra-neighbourhood trips with relatively low per-trip fares, whereas the highest total-revenue pickup zone (JFK Airport, Section 10) derives its revenue from a combination of high volume and a much higher average fare per trip ($63.73) driven by longer average distances. Frequency and profitability are correlated but not identical: airport routes are comparatively rare relative to short Manhattan hops, yet generate disproportionately more revenue per trip.

j) What percentage of records contain potential anomalies?

Approximately 0.40% of the already-cleaned dataset (33,731 of 8,470,368 records) show an extreme fare-per-mile value exceeding $50/mile. Separately, at the data-cleaning stage, 7.86% of the original raw dataset (722,848 of 9,193,216 records) were flagged and removed for more fundamental data-quality issues (missing fields, invalid passenger counts, zero/negative distances, invalid fares, bad timestamps, or impossible durations).

k) What transportation insights can management derive from the analysis?

- Demand is highly time-dependent: fleet/driver allocation should scale up through the day toward a 6 PM peak and scale down sharply overnight (4 AM sees 13× fewer trips than the peak hour).

- Airport service (JFK especially) is disproportionately valuable: it represents a small fraction of total trip volume but a very large share of total revenue, making airport-zone driver availability a high-value operational priority.
- Short trips dominate by volume but are the least revenue-efficient per trip; a strategy that also captures more 10+ mile trips (airport runs, outer-borough travel) would improve average revenue per trip without needing to grow total trip volume.
- Credit-card adoption is already very high (81% of trips); further reducing friction for the remaining cash/dispute/no-charge trips is a smaller but achievable revenue-integrity opportunity.
- The fare-per-mile anomaly rate (0.4%) and the cleaning-stage data-quality issue rate (7.9%) both represent measurable targets for either fraud/error monitoring or improvements to how trip data is captured at the point of sale.

l) When does Hadoop MapReduce provide a meaningful advantage over conventional processing?

Based on the direct comparison in Section 12, Hadoop's advantage is not visible at this project's scale (≈8.5 million records fit comfortably in a single machine's memory, where Pandas wins on raw speed). Hadoop's advantage emerges once dataset size exceeds what a single machine can hold in memory or process in reasonable time at that point Pandas either fails outright (out-of-memory) or requires substantial manual engineering (chunked reading, external merge-sort aggregation) to keep working, while Hadoop's distributed model, and specifically running the same code across many physical nodes rather than one, allows throughput to scale roughly linearly by adding cluster capacity, with fault tolerance (automatic re-execution of failed tasks, as was observed first-hand during this project's own shuffle-failure incidents) built in as a byproduct of the architecture.

14. Limitations

- Single-node deployment: this project ran on one physical machine acting as both NameNode/DataNode and ResourceManager/NodeManager, so the true multi-node parallelism and fault-tolerance benefits of Hadoop (e.g. surviving the loss of an entire physical node) could not be demonstrated, only inferred from the architecture.
- Native Windows Hadoop: Hadoop is primarily developed and documented for Linux; running natively on Windows introduced avoidable friction (path-separator issues with hdfs dfs -put, and shuffle-phase HTTP fetch failures under load) that a Linux or WSL-based deployment would likely have avoided.
- Reducer parallelism: no job explicitly set -D mapreduce.job.reduces, so all reducers defaulted to a single reducer task, meaning the reduce phase itself was never parallelised across multiple reducers in this project only the map phase benefited from multiple concurrent tasks.
- Anomaly detection was run against already-cleaned data rather than the raw data its logic was originally designed for, which suppressed six of its seven planned anomaly categories (they had already been filtered out during cleaning); a more complete anomaly analysis would additionally run this mapper against /taxi_project/input/raw/ directly.
- Route-level revenue figures (Section 9g) and exact daily-demand counts (Section 9b) were sourced from the generated chart visualisations rather than from a saved raw text export of the final full-dataset job output, so they are reported as approximate rather than exact-to-the-cent figures.

15. Conclusion

This project implemented a complete, working, end-to-end Hadoop-based analytics pipeline for NYC Yellow Taxi trip data: HDFS storage with a documented directory structure, a documented cleaning stage that quantifies rather than silently discards data-quality issues, nine independent Python MapReduce analyses covering demand, location, revenue, payment, distance, route and duration patterns plus anomaly detection, a compulsory two-stage MapReduce pipeline chaining HDFS output directly into a second job's input, YARN-monitored execution across 28 real submitted applications (including genuine debugging of real failures), and a head-to-head performance comparison against conventional Pandas processing. The results directly answer the case study's business questions and identify concrete, data-backed operational recommendations most notably, the outsized revenue importance of airport pickup zones and the strongly time-of-day-dependent nature of demand.

16. References

- NYC Taxi & Limousine Commission. TLC Trip Record Data. https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page
- Apache Software Foundation. Apache Hadoop 3.5.0 Documentation. https://hadoop.apache.org/docs/r3.5.0/
- Apache Software Foundation. Hadoop Streaming Documentation. https://hadoop.apache.org/docs/r3.5.0/hadoop-streaming/HadoopStreaming.html
- McKinney, W. pandas: powerful Python data analysis toolkit. https://pandas.pydata.org/
- Kumar, Dr. Kundan. Big Data Essentials Individual Assignment Brief: Distributed Taxi Trip Analytics Using Apache Hadoop, HDFS and Python MapReduce. AUCA, 2026.

17. Deliverables

The following are submitted alongside this report:

- Source code: all Python mapper and reducer programs (mappers/, reducers/), plus clean_taxi_data.py, convert_parquet_to_csv.py, perf_compare_pandas.py, and make_visualizations.py available at [INSERT GITHUB / GOOGLE DRIVE LINK HERE]
- commands.txt the full set of HDFS and Hadoop Streaming commands needed to reproduce this work
- README.md environment assumptions and step-by-step execution instructions
- This report (PDF), including all required screenshots inserted at the marked locations